

TP3 Team Norms

Updated from TP2 Team Norms with track changes for TP3

Team: Sutton Harr, Rufat Babayev, Ahad Mammadli

Review note: Changed norm text below uses tracked changes. Each section also includes a short 'What prompted the change' note so the reason for the update is easy to grade and edit later.

Coding Style, Architecture, and Reuse Consistency:

Keep new TP3 code aligned with the provided FoundationsS26 structure and with the team's existing TP2 package boundaries, but also document architectural intent before parallel implementation begins. When adding staff-facing functionality, reuse existing services, entities, dialogs, and patterns where practical instead of creating one-off flows that duplicate logic.

Keep methods focused and readable, separate controller, service, database, and analytics responsibilities cleanly, and add short internal comments only when they explain why the code was written this way. If a class or method supports a new staff story, design element, or test case, leave a nearby trace note so the Javadoc, design PDFs, and screencasts can all point to the same implementation.

What prompted the change: TP2 mostly expanded student-facing functionality, but TP3 adds architecture and detailed design deliverables plus more staff-oriented features. We needed the norm to cover reuse, design consistency, and explicit links between code, stories, and tests.

Meetings, Communication, Backlog, and Status Tracking:

Meet at least twice per week during active TP3 work with at least 47 hours between standups, decide the next meeting before ending the current one, and post an asynchronous update the same day if attendance changes. Each member should be ready to show backlog status, what was completed, what is next, and what is blocked, because the standups now drive the required status summary recordings.

All project work should live in GitHub and on the shared backlog board with a priority, owner, status, due date, and link to the related branch, commit, or pull request when possible. If priorities or due dates change, update the board and the current plan document the same day so the written plan, recordings, and actual work never drift apart.

What prompted the change: TP2 required standup-style evidence, but TP3 makes backlog management and repeated status summaries a major graded artifact. The norm now emphasizes schedule spacing, visible ownership, and same-day plan updates.

Documentation, Javadoc, and Requirement or Test Traceability:

Document TP3 work will be done while it is being built rather than after the code settles. User stories, architecture decisions, detailed design notes, Javadocs, internal comments, screenshots, and manual-test evidence should be captured incrementally so each new staff feature already has documentation material by the time the PDFs are assembled.

What prompted the change: TP2 already taught us that documentation cannot be left until the end. TP3 raises the bar further by requiring Architecture, Detailed Design, New Code Javadoc, Test Code Javadoc, and explicit evidence of which tests validate which methods.

UI/UX, Staff Workflow, and Demonstration Readiness:

Maintain overall similarity to the provided application and the existing TP2 screens so new staff-facing pages feel like part of the same system.

Labels, filters, tables, action buttons, alerts, and empty-state messages should stay consistent across admin, discussion, report-review, and analytics-style workflows. New screens should also be designed with realistic demonstrations in mind: important outputs should be readable on screen, realistic data should be easy to load or copy in, and the flow for staff tasks such as reviewing reports, checking participation, or managing user state should be understandable to non-technical viewers during the execution screencast.

What prompted the change: TP2 focused mostly on student interaction quality. TP3 still needs that consistency, but now the project must also demonstrate staff workflows clearly and professionally in screencasts aimed at graders and prospective employers.

Testing, Refactoring, and Release Readiness:

A backlog item is not Done until the feature runs, its required automated and manual tests have been updated or created, affected documentation has been refreshed, and the team has enough saved evidence to support the final PDFs and screencasts. High-risk or unclear features should be prototyped and tested early rather than deferred to final polish.

When implementation reveals a flaw in earlier assumptions, refactor early and then update the related requirements, design notes, tests, and plan rather than building more code on top of a shaky base. Before submission, each PDF, recording, archive, and Javadoc folder should have a primary owner, a second reviewer, and a final verification pass by the whole team because TP3 makes submission quality and alignment a shared responsibility.

What prompted the change: TP2 showed that working code is not the same as a ready submission. HW3 also highlighted the value of early prototypes and targeted testing, and TP3 explicitly expects refactoring, alignment, and professionally documented test evidence.